

Modelos Declarativos

Objetivos de Aprendizaje

Al finalizar este tema, serás capaz de:

-  Definir modelos como clases Python
-  Usar tipos de datos SQLAlchemy correctamente
-  Configurar primary keys, defaults y constraints
-  Aplicar el nuevo estilo de SQLAlchemy 2.0

Contenido

1. Anatomía de un Modelo

Un modelo SQLAlchemy representa una tabla en la base de datos:

```
from datetime import datetime
from sqlalchemy import String
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column

class Base(DeclarativeBase):
    pass

class User(Base):
    # Nombre de la tabla en la base de datos
    __tablename__ = "users"

    # Columnas con type hints (SQLAlchemy 2.0)
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100))
    email: Mapped[str] = mapped_column(String(255), unique=True)
    age: Mapped[int | None] = mapped_column(default=None)
    is_active: Mapped[bool] = mapped_column(default=True)
    created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)
```

Componentes Clave

Componente	Propósito
<code>__tablename__</code>	Nombre de la tabla SQL
<code>Mapped[T]</code>	Declara el tipo Python del atributo
<code>mapped_column()</code>	Configura la columna SQL

2. Tipos de Datos

Mapeo Python → SQLAlchemy → SQL

Python Type	SQLAlchemy	SQL (SQLite)	SQL (PostgreSQL)
<code>int</code>	<code>Integer</code>	<code>INTEGER</code>	<code>INTEGER</code>

str	String(n)	VARCHAR(n)	VARCHAR(n)
str	Text	TEXT	TEXT
float	Float	REAL	DOUBLE PRECISION
bool	Boolean	INTEGER	BOOLEAN
datetime	DateTime	TIMESTAMP	TIMESTAMP
date	Date	DATE	DATE
bytes	LargeBinary	BLOB	BYTEA
Decimal	Numeric	NUMERIC	NUMERIC

```

from datetime import datetime, date
from decimal import Decimal
from sqlalchemy import String, Text, Numeric, LargeBinary
from sqlalchemy.orm import Mapped, mapped_column

class Product(Base):
    __tablename__ = "products"

    id: Mapped[int] = mapped_column(primary_key=True)

    # String con longitud máxima
    name: Mapped[str] = mapped_column(String(200))

    # Texto largo sin límite
    description: Mapped[str | None] = mapped_column(Text, default=None)

    # Decimal para precios (precisión importante)
    price: Mapped[Decimal] = mapped_column(Numeric(10, 2))

    # Entero
    stock: Mapped[int] = mapped_column(default=0)

    # Booleano
    is_available: Mapped[bool] = mapped_column(default=True)

    # Fechas
    created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)
    release_date: Mapped[date | None] = mapped_column(default=None)

```

3. Tipos Opcionales (Nullable)

```

from sqlalchemy.orm import Mapped, mapped_column

class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)

    # Requerido (NOT NULL) - sin | None

```

```

email: Mapped[str] = mapped_column(String(255), unique=True)

# Opcional (NULL permitido) - con | None
phone: Mapped[str | None] = mapped_column(String(20), default=None)
bio: Mapped[str | None] = mapped_column(Text, default=None)

# También opcional
age: Mapped[int | None] = mapped_column(default=None)

```

El type hint determina si la columna es nullable:

- `Mapped[str]` → NOT NULL
- `Mapped[str | None]` → NULL permitido

4. Primary Keys

```

from sqlalchemy.orm import Mapped, mapped_column

class User(Base):
    __tablename__ = "users"

    # Autoincrement por defecto con integer primary key
    id: Mapped[int] = mapped_column(primary_key=True)

# Primary key con UUID
import uuid
from sqlalchemy import Uuid

class Session(Base):
    __tablename__ = "sessions"

    id: Mapped[uuid.UUID] = mapped_column(
        Uuid,
        primary_key=True,
        default=uuid.uuid4
    )

```

5. Valores por Defecto

```

from datetime import datetime
from sqlalchemy import func
from sqlalchemy.orm import Mapped, mapped_column

class Article(Base):
    __tablename__ = "articles"

    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(200))

    # Default Python (se evalúa en Python)
    is_published: Mapped[bool] = mapped_column(default=False)

    # Default con función (se evalúa al insertar)

```

```

created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)

# Default SQL (lo ejecuta la base de datos)
updated_at: Mapped[datetime] = mapped_column(
    server_default=func.now(), # SQL: DEFAULT CURRENT_TIMESTAMP
    onupdate=datetime.utcnow # Se actualiza en cada UPDATE
)

# Default con valor fijo
views: Mapped[int] = mapped_column(default=0)
status: Mapped[str] = mapped_column(String(20), default="draft")

```

default vs server_default

```

# default - Python genera el valor antes de INSERT
created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)
# INSERT INTO articles (created_at, ...) VALUES ('2024-01-15 10:30:00', ...)

# server_default - La base de datos genera el valor
created_at: Mapped[datetime] = mapped_column(server_default=func.now())
# INSERT INTO articles (...) VALUES (...) -- DB usa CURRENT_TIMESTAMP

```

6. Constraints (Restricciones)

```

from sqlalchemy import String, CheckConstraint, UniqueConstraint
from sqlalchemy.orm import Mapped, mapped_column

class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)

    # Unique - valor único en toda la tabla
    email: Mapped[str] = mapped_column(String(255), unique=True)

    # Index - mejora búsquedas
    username: Mapped[str] = mapped_column(String(50), unique=True, index=True)

    age: Mapped[int | None] = mapped_column(default=None)
    role: Mapped[str] = mapped_column(String(20), default="user")

    # Constraints a nivel de tabla
    __table_args__ = (
        # Check constraint - validación SQL
        CheckConstraint("age >= 0 AND age <= 150", name="valid_age"),
        CheckConstraint("role IN ('user', 'admin', 'moderator')", name="valid_role"),

        # Unique constraint compuesto
        UniqueConstraint("email", "role", name="unique_email_per_role"),
    )

```

7. Ejemplo Completo: Sistema de Blog

```
"""
models.py - Modelos para un sistema de blog
"""

from datetime import datetime
from sqlalchemy import String, Text, ForeignKey, func
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column

class Base(DeclarativeBase):
    pass

class Author(Base):
    """Modelo de autor"""
    __tablename__ = "authors"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100))
    email: Mapped[str] = mapped_column(String(255), unique=True, index=True)
    bio: Mapped[str | None] = mapped_column(Text, default=None)
    is_active: Mapped[bool] = mapped_column(default=True)
    created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)

    def __repr__(self) -> str:
        return f"Author(id={self.id}, name={self.name!r})"

class Post(Base):
    """Modelo de publicación"""
    __tablename__ = "posts"

    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(200))
    slug: Mapped[str] = mapped_column(String(250), unique=True, index=True)
    content: Mapped[str] = mapped_column(Text)
    summary: Mapped[str | None] = mapped_column(String(500), default=None)

    # Foreign key (lo veremos en detalle en la siguiente semana)
    author_id: Mapped[int] = mapped_column(ForeignKey("authors.id"))

    # Estados y metadata
    is_published: Mapped[bool] = mapped_column(default=False)
    views: Mapped[int] = mapped_column(default=0)

    # Timestamps
    created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)
    updated_at: Mapped[datetime | None] = mapped_column(
        default=None,
        onupdate=datetime.utcnow
    )
    published_at: Mapped[datetime | None] = mapped_column(default=None)

    def __repr__(self) -> str:
```

```
return f"Post(id={self.id}, title={self.title!r})"
```

```
class Tag(Base):
    """Modelo de etiqueta"""
    __tablename__ = "tags"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(50), unique=True)
    slug: Mapped[str] = mapped_column(String(60), unique=True, index=True)

    def __repr__(self) -> str:
        return f"Tag(id={self.id}, name={self.name!r})"
```

8. Crear las Tablas

```
# main.py o script separado
from database import engine, Base
from models import Author, Post, Tag # Importar todos los modelos

# Crear todas las tablas
Base.metadata.create_all(bind=engine)
print("✅ Tablas creadas")

# Ver las tablas creadas
for table in Base.metadata.tables:
    print(f" - {table}")
```

Verificación

```
# Verificar que los modelos están bien definidos
from models import Author, Post, Tag
from database import engine, Base

# Esto muestra el SQL que se generaría
from sqlalchemy.schema import CreateTable

print(CreateTable(Author.__table__).compile(engine))
```

✅ Checklist

- ☐ Sé definir modelos con `__tablename__`
- ☐ Uso `Mapped[T]` y `mapped_column()` correctamente
- ☐ Entiendo los tipos de datos y su mapeo
- ☐ Sé configurar defaults y constraints
- ☐ Puedo crear las tablas con `create_all()`

[← Anterior: Configuración](#) | [Siguiente: Operaciones CRUD →](#)